

Activity: Complete a SQL join

Scenario

In this scenario, you'll investigate a recent security incident that compromised some machines.

You are responsible for getting the required information from the database for the investigation.

Task 1. Match employees to their machines

First, you must identify which employees are using which machines. The data is located in the `machines` and `employees` tables.

You must use a SQL inner join to return the records you need based on a connecting column. In the scenario, both tables include the `device_id` column, which you'll use to perform the join.

1. Run the following query to retrieve all records from the `machines` table:

```
SELECT *  
  
FROM machines;
```

```
MariaDB [organization]> SELECT *  
->  
-> FROM machines;
```

device_id	operating_system	email_client	OS_patch_date	employee_id
a184b775c707	OS 1	Email Client 1	2021-09-01	1156
a192b174c940	OS 2	Email Client 1	2021-06-01	1052
a305b818c708	OS 3	Email Client 2	2021-06-01	1182
a317b635c465	OS 1	Email Client 2	2021-03-01	1130
a320b137c219	OS 2	Email Client 2	2021-03-01	1000
a398b471c573	OS 3	Email Client 2	2021-12-01	0
a667b270c984	OS 1	Email Client 1	2021-03-01	1078
a821b452c176	OS 2	Email Client 2	2021-12-01	1104
a998b568c863	OS 3	Email Client 1	2021-12-01	1026
b157c491d493	OS 2	Email Client 1	2021-03-01	0
b239c825d303	OS 1	Email Client 1	2021-03-01	1001
b264c773d977	OS 2	Email Client 2	2021-03-01	1157
b265c937d713	OS 2	Email Client 1	2021-09-01	1131

You'll note that this query is not sufficient to perform the join and retrieve the information you need. If you look at the results of that query, you will see a list of hardware (like `device_id` and `operating_system`), but you won't see any **employee names**.

- **Table machines:** Has the machine info, but doesn't know who owns it.
- **Table employees:** Has the person's name, but doesn't know their laptop specs.

The Solution: Using INNER JOIN

To get the full story (Machine + Name), you have to use a **JOIN** to bridge the gap.

```
SELECT *
FROM machines                    -- 1. Start with the hardware
INNER JOIN employees             -- 2. Pull in the people
ON machines.device_id = employees.device_id; -- 3. Match them by ID
```

Why this works:

1. **FROM:** Tells SQL to open the "Machines" table.
2. **INNER JOIN:** Tells SQL to open the "Employees" table and bring it over.
3. **ON:** Tells SQL to look at the `device_id` in both tables. If the IDs match, it glues those two rows together into one complete record.

An INNER JOIN is like a puzzle. It only keeps the pieces that fit together perfectly.

- If a row has a match in both tables, it stays.
- If a row has no match, it is hidden.

In an INNER JOIN, unmatched rows disappear. Therefore, you pick one table to be the "Master List" so no data is lost from it.

To join two tables, SQL needs a "common link" and a way to avoid confusion.

The Shared Connection: device_id

The `device_id` is the **Shared Key**. It is the "bridge" because it is the only piece of information that exists in both tables.

- The **machines** table has it.
- The **employees** table has it.

Without this common column, SQL has no way to know which machine belongs to which person. It is the only piece of information that exists in both "piles" of data.

The Precise Address: Dot Notation

Since both tables have a column named `device_id`, SQL will get confused if you don't specify which is which. The **dot** provides a full address (Table.Column):

- `machines.device_id`: "Look in the **machines** table."
- `employees.device_id`: "Look in the **employees** table."

Using the table name before the dot ensures SQL matches the correct columns without any "ambiguity" errors.

The Syntax

Think of the query as a three-step instruction:

1. **FROM**: Where you start (the first table).
2. **INNER JOIN**: What you're adding (the second table).
3. **ON**: How they connect (the Table.Column match).

2. Complete the query to perform an inner join between the `machines` and `employees` tables on the `device_id` column. Replace X and Y with this column name:

```
MariaDB [organization]> SELECT *
->
-> FROM machines
->
-> INNER JOIN employees ON machines.employee_id = employees.employee_id;
```

device_id	operating_system	email_client	OS_patch_date	employee_id	emp
a184b775c707	OS 1	Email Client 1	2021-09-01	1156	
1001171-010	OS 2	Email Client 1	2021-06-01	1050	

How many rows did the inner join return?

☒ 185

☐ 132

☐ 124

☐ 85

Submit

```
| z566a147b347 | OS 1 | Email  
| z654a154b259 | OS 2 | Email  
| z803a233b718 | OS 1 | Email  
| z942a966b589 | OS 3 | Email  
+-----+-----+-----+  
185 rows in set (0.229 sec)  
  
MariaDB [organization]>
```

The lab uses `SELECT *` (Select All) for simplicity. When you join two tables that both have a `device_id` column, `SELECT *` will return **two columns** with the exact same name and data. In real-world work, you should specify the tables for clarity:

```
=>  
-> INNER JOIN employees ON machines.employee_id = employees.employee_id;  
+-----+-----+-----+-----+-----+-----+  
device_id | operating_system | email_client | OS_patch_date | employee_id | employee_id | c  
+-----+-----+-----+-----+-----+-----+  
a184b775c707 | OS 1 | Email Client 1 | 2021-09-01 | 1156 | 1156 | a  
a192b174c940 | OS 2 | Email Client 1 | 2021-06-01 | 1052 | 1052 | a  
+-----+-----+-----+-----+-----+-----+  
  
SELECT machines.device_id, employees.username, machines.operating_system
```

This prevents ambiguity and makes your report much easier to read.

```
MariaDB [organization]> SELECT machines.device_id, employees.username, machines.operating_system
-> FROM machines
-> INNER JOIN employees ON machines.employee_id = employees.employee_id;
```

device_id	username	operating_system
a184b775c707	dellery	OS 1
a192b174c940	jdarosa	OS 2
a305b818c708	mmora	OS 3
a317b635c465	tsnow	OS 1
a320b137c219	elarson	OS 2

latest

Task 2. Return more data

You now must return the information on all machines and the employees who have machines. Next, you must do the reverse and retrieve the information of all employees and any machines that are assigned to them.

To achieve this, you'll complete a left join and a right join on the `employees` and `machines` tables. The results will include all records from one or the other table. You must link these tables using the common `device_id` column.

The Syntax: Left vs. Right

Before you pick a join type, look at the order of the tables in your code:

- **Left Table:** The table listed first (immediately after `FROM`).
- **Right Table:** The table listed second (immediately after `JOIN`).

Think of **Left** and **Right** joins as a way to find what's missing: they show you machines that aren't assigned to anyone and people who don't have a device.

1. LEFT JOIN (Keep everything on the Left)

This tells SQL to keep every row from the **first** table, even if there is no match in the second.

- **The Goal:** See every single machine, even if it's sitting in a closet unassigned.
- **The Result:** If a machine has no user, the employee columns will simply say **NULL**.

2. RIGHT JOIN (Keep everything on the Right)

This tells SQL to keep every row from the **second** table, even if there is no match in the first.

- **The Goal:** See every single employee, even if they haven't been issued a laptop yet.
- **The Result:** If an employee has no machine, the machine columns will say **NULL**.

1. Run the following SQL query to connect the `machines` and `employees` tables through a left join. You must replace the key word X in the query:

```
MariaDB [organization]> SELECT *
->
-> FROM machines
->
-> LEFT JOIN employees ON machines.device_id = employees.device_id;
```

device_id	operating_system	email_client	OS_patch_date	employee
a320b137c219	OS 2	Email Client 2	2021-03-01	1

What is the value in the username column for the last record returned?

- ☐ asundara
- ☐ areyes
- ☒ NULL
- ☐ cgriffin

Submit

Task 3. Retrieve login attempt data

To continue investigating the security incident, you must retrieve the information on all employees who have made login attempts. To achieve this, you'll perform an inner join on the `employees` and `log_in_attempts` tables, linking them on the common `username` column.

- Run the following SQL query to perform an inner join on the `employees` and `log_in_attempts` tables. Replace X with the name of the right table. Then replace Y and Z with the name of the column that connects the two tables:

```
MariaDB [organization]> SELECT *  
->  
-> FROM employees  
->  
-> INNER JOIN log_in_attempts ON employees.username = log_in_attempts.username;  
+-----+-----+-----+-----+-----+-----+  
| employee_id | device_id | username | department | office | event_id |  
+-----+-----+-----+-----+-----+-----+  
| 1032 | c773b303i639 | irafael | Information Technology | Central-309 | 1
```

How many records are returned by this inner join?

☐ 210

☐ 175

☒ 200

☐ 145

Submit